



# ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 4 – CODE SMELLS

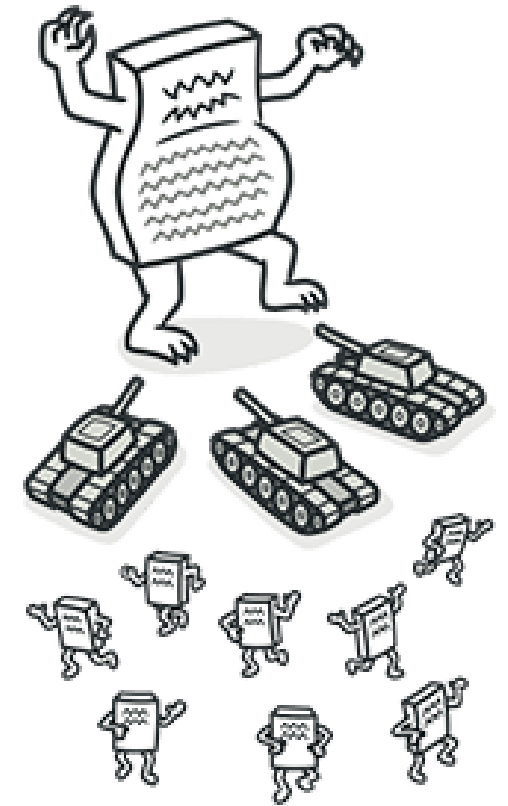


# CODE SMELLS

- Son indicadores de posibles problemas en el código
- Generalmente indican que debe refactorizarse el código

# BLOATERS

- Algo que creció tanto que es complejo de entender y manejar
- No se identifican inmediatamente, sino que se acumulan (hasta que se vuelve inmanejable)



# BLOATERS

## LONG METHODS

```
showChapter(chapter, tutorial) {
  if(chapter.orderIndex == 1) {
    this.redirectToRoot()
  }

  let higherChapters = chapter.items.filter(item => item.higher)
  let lowerTutorials = chapter.items.filter(item => !item.higher)

  let title = chapter.title
  let description = chapter.description

  if(chapter.lowerItem) {
    this.nextPath = this.getChapterPath(this.tutorial, chapter.lowerItem)
    this.nextLink = this.nextPath
  } else if (!!tutorial.pathId && !!lowerTutorials) {
    this.nextPath = this.getTutorialPath(lowerTutorials[0])
    this.nextLink = this.nextPath
  } else if (chapter.last && !tutorial.footLinks.length > 0) {
    this.nextPath = tutorial.footLinks.url
  }

  if(higherChapters.length === 0) {
    this.prevLink = this.getChapterPath(tutorial, higherChapters)
  } else if (!!tutorial.pathId && tutorial.higherItem) {
    let higherTutorial = tutorial.higherItem
    lastChapter = higherTutorial.chapters[higherTutorial.chapters.length - 1]

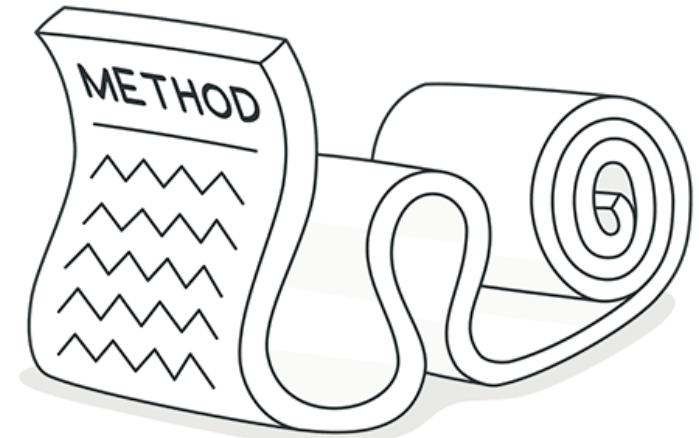
    if (lastChapter.orderIndex == 1) {
      this.prevLink = this.getTutorialPath(higherTutorial)
    } else {
      this.prevLink = this.getChapterPath(higherTutorial, higherChapter)
    }
  }

  this.storeProps = {
    checkableType: 'Chapter',
    checkableId: chapter.id,
    checkboxes: this.signedIn ? currentUser.getCheckboxesFor(chapter) : []
  }

  this.setStore()

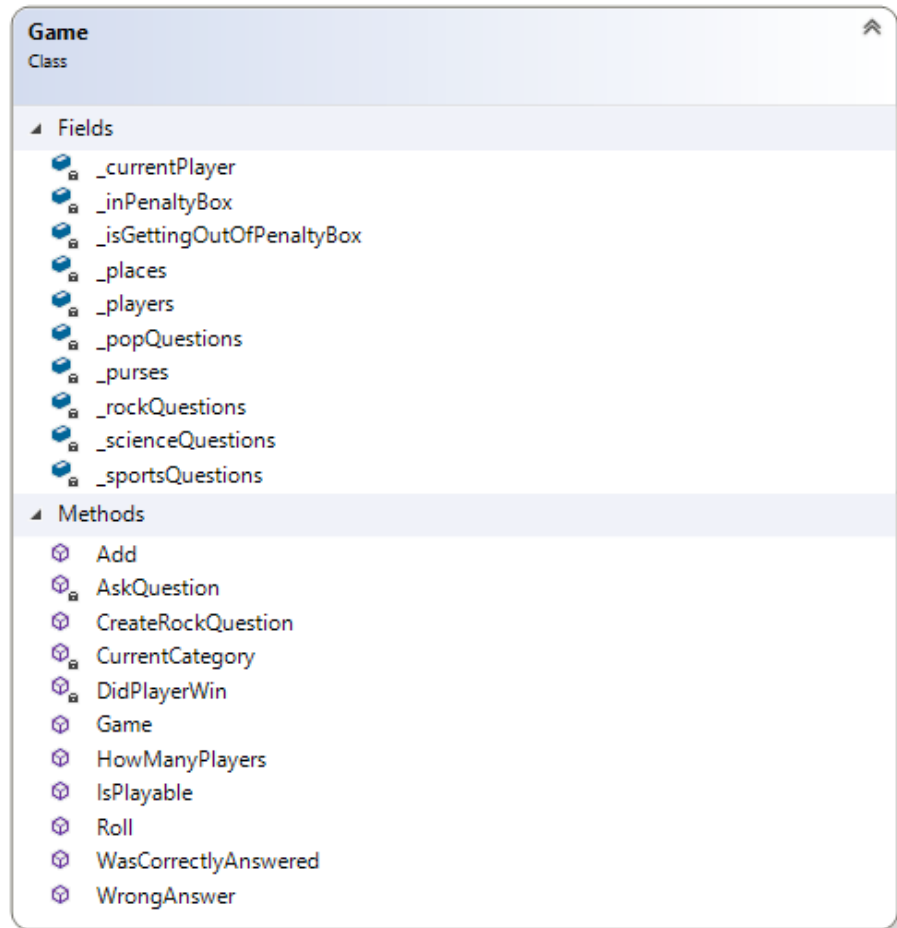
  this.modal = this.getModalChapter(chapter) || this.getModalTutorial(chapter)
}
```

- **Síntoma:** un método tiene muchas líneas de código
- **Razones:** siempre se agregan cosas, pero nunca se sacan
- **Remedio:** si se siente la necesidad de explicar en un comentario lo que se hace dentro de un método, se debería extraer en uno nuevo. (¡Elegir un nombre descriptivo!)

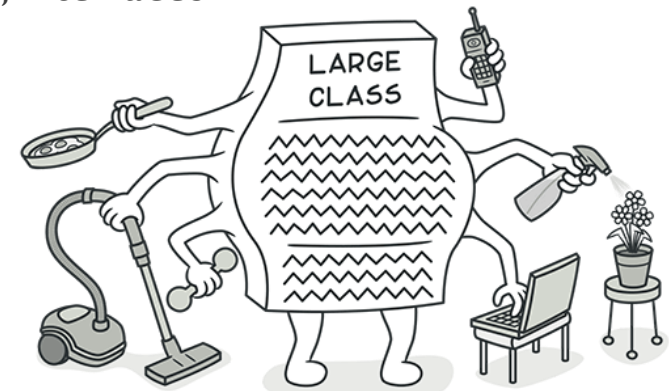


# BLOATERS

## LARGE CLASS



- **Síntoma:** una clase tiene demasiados atributos/métodos/líneas de código
- **Razones:** las clases suelen ser pequeñas inicialmente, pero a medida que el sistema evoluciona las mismas crecen
- **Remedio:** extraer clases, subclases, interfaces



# BLOATERS

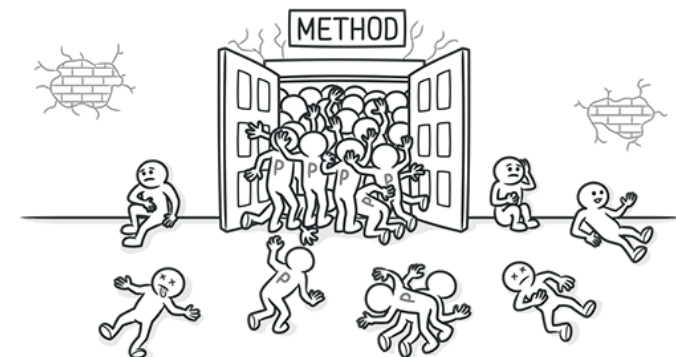
## LONG PARAMETER LIST

```
void paint  
(Graphics gr, double x, double y,  
double width, double height,  
bool shouldValidate)  
{.....}
```

Caller side:

```
public void someMethod(Graphics gr)  
{  
    double x = 100;  
    double y = 200;  
    double width = 100;  
    double height = width/2;  
    paint(gr, x, y, width, height, true);  
}
```

- **Síntoma:** un método posee más de tres o cuatro parámetros
- **Razones:** varios algoritmos se combinaron en un único método, no se abstraio alguna entidad del dominio, se intentó independizar clases
- **Remedio:** reemplazar parámetros por llamada a clase enviando un objeto entero, o juntar parámetros en un objeto

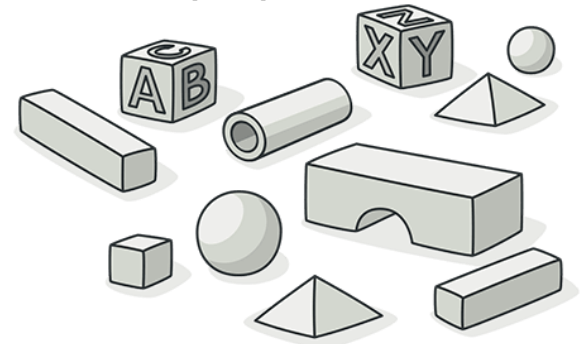


# BLOATERS

## PRIMITIVE OBSESSION

```
public class CheckingAccount
{
    public int AccountNumber { get; set; }
    public string CustomerName { get; set; }
    public string Email { get; private set; }
    public string Address { get; set; }
    public int ZipCode { get; set; }
    public string City { get; set; }
    public string State { get; set; }
    public string Country { get; set; }
    public string SocialSecurityNumber { get; set; }
    public DateTime ActiveDate { get; set; }
    public string GetSSNLast4Digit()...
}
```

- **Síntoma:** se usan primitivos en lugar de objetos pequeños para tareas sencillas, se codifica información en constantes, se usan cadenas constantes como nombres en campos de datos
- **Razones:** se utilizan para simular tipos en lugar de declarar un tipo separado, es más sencillo
- **Remedio:** agrupar primitivos en su propia clase



# DISPENSABLES

- Algo innecesario, cuya ausencia hace que el código se vea limpio

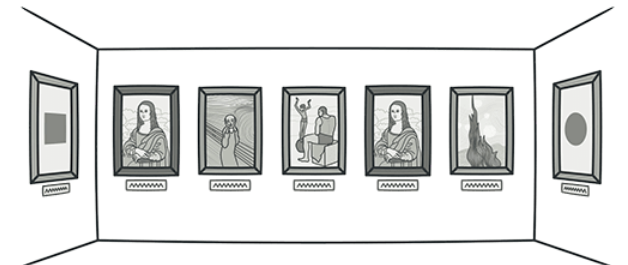




# DISPENSABLES DUPLICATE CODE

```
function calculateTax(subtotal, country, state, taxrates) {  
  let taxrate;  
  if(country === 'US') {  
    taxrate = taxrates[state];  
  } else {  
    taxrate = taxrates[country];  
  }  
  return subtotal + subtotal * taxrate;  
}  
  
function findTimeZone(country, state, zones) {  
  let timezone;  
  if(country === 'US') {  
    timezone = zones[state];  
  } else {  
    timezone = zones[country];  
  }  
  return timezone;  
}
```

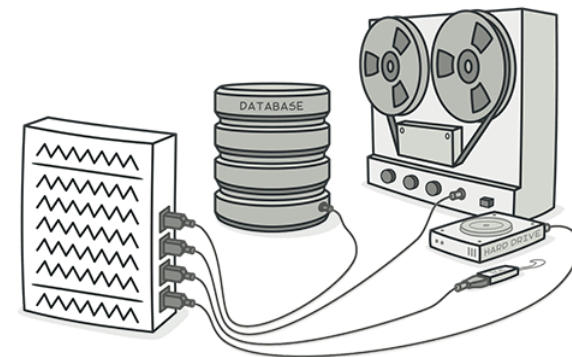
- **Síntoma:** dos fragmentos de código se parecen o son casi iguales
- **Razones:** varios programadores que trabajan en distintas partes del código al mismo tiempo, hay apuro y el copiar y pegar no se ve tan malo
- **Remedio:** extraer en un único método (si el código duplicado pertenece a la misma clase), mover el método a la clase base (si son dos subclases), crear una clase base o extraer una clase aparte (si son clases distintas)



# DISPENSABLES DATA CLASS

| DataClass                                                                                               |
|---------------------------------------------------------------------------------------------------------|
| fieldA                                                                                                  |
| fieldB                                                                                                  |
| getFieldA()<br>getFieldB()<br>setFieldA()<br>setFieldB()<br>useFieldA()<br>useFieldB()<br>useFieldsAB() |

- **Síntoma:** una clase que contiene campos y getters y setters únicamente
- **Razones:** encapsulamos un tipo en una clase aparte, pero no tiene comportamiento específico con su estado
- **Remedio:** buscar en código cliente si la funcionalidad podría estar en la clase



# DISPENSABLES

## LAZY CLASS

```
class contactUs
{
    public function addressUsa()
    {
        $address = new Array();
        $address['streetNo'] = 2074;
        $address['streetName'] = 'JFK street';
        $address['zipCode'] = '507874';

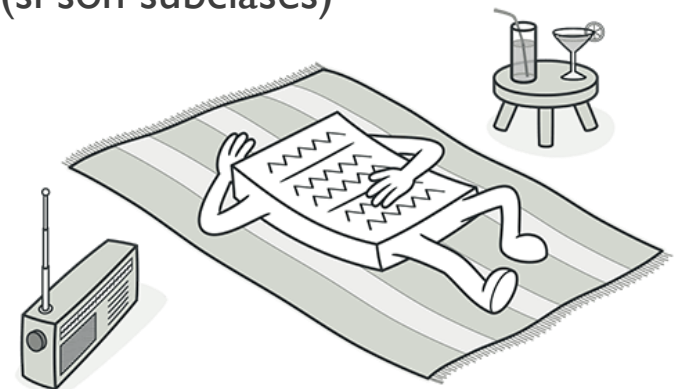
        return $address['streetName']. ' '. $address['streetNo']. ' ', ' '. $address['zipCode'];
    }

    public function addressGermany()
    {
        $address = new Array();
        $address['streetNo'] = '25';
        $address['streetName'] = 'Frankfurter str.';
        $address['zipCode'] = '80256';

        return $address['streetName']. ' '. $address['streetNo']. ' ', ' '. $address['zipCode'];
    }

    public function hotLine(){
        return '+49 01687 000 000';
    }
}
```

- **Síntoma:** una clase no nos llama la atención
- **Razones:** se diseñó para ser funcional, pero se convirtió en una clase muy pequeña; o se diseñó para dar soporte a una funcionalidad futura que nunca llegó
- **Remedio:** “trasplantar” la funcionalidad a otra clase, o incluir la funcionalidad en la clase base (si son subclases)



# DISPENSABLES DEAD CODE

- **Síntoma:** una clase, parámetro o campo no se usa más
- **Razones:** los requerimientos cambian, pero no se limpia el código
- **Remedio:** buscar en código el código que no es utilizado (¡un buen IDE!)



# COUPLERS

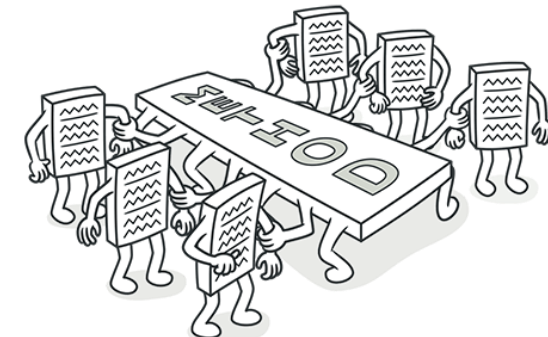
- Practicas que aumentan excesivamente el acoplamiento entre clases



# COUPLERS INAPPROPRIATE INTIMACY

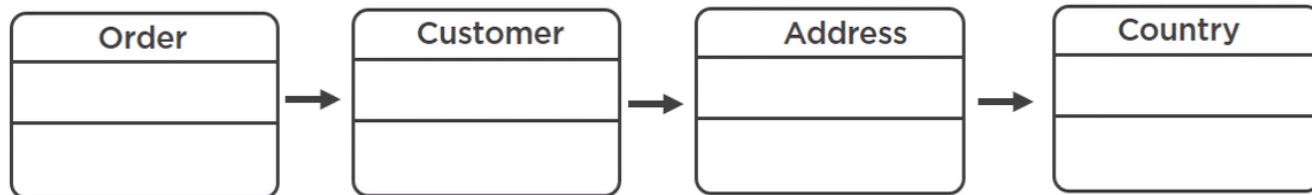
```
class User {  
    function notify($userName, UserContactDetails $contactDetails, $message) {  
        $messageBody = 'Dear ' . $userName;  
        $messageBody .= ' ( ' . $contactDetails->getStreetNumber() . ' ' .  
            $contactDetails->getStreetName() . ' ). ' . $message;  
        $messageBody .= $message;  
  
        $this->notificationService->sendSMS($contactDetails->getPhoneNubmer(), $messageBody);  
    }  
}
```

- **Síntoma:** una clase utiliza los métodos y atributos internos de otra clase
- **Razones:** las clases trabajan en conjunto frecuentemente
- **Remedio:** mover los métodos y atributos internos de la clase B a la A (si la clase A no los necesita), realizar invocaciones a través de métodos

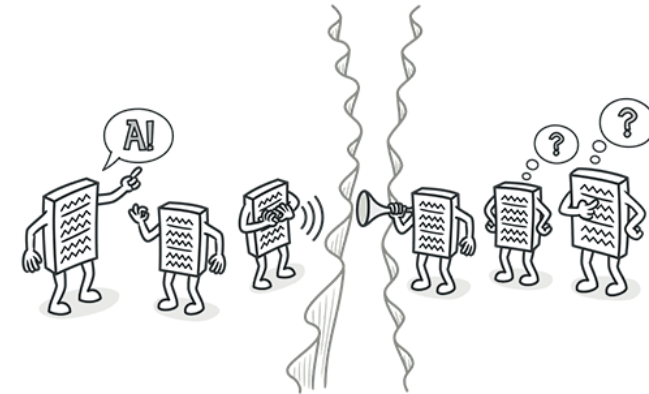


# COUPLERS

## MESSAGE CHAINS

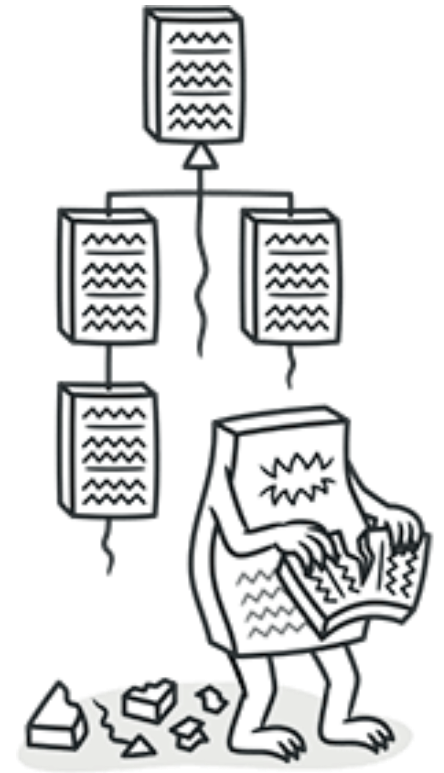


- **Síntoma:** se observan líneas de código del estilo `a.getB().getC().getD()`
- **Razones:** las clases dependen de navegar a través de la estructura de clases
- **Remedio:** ocultar la dependencia delegando a otro método la obtención de datos



# OBJECT-ORIENTATION ABUSERS

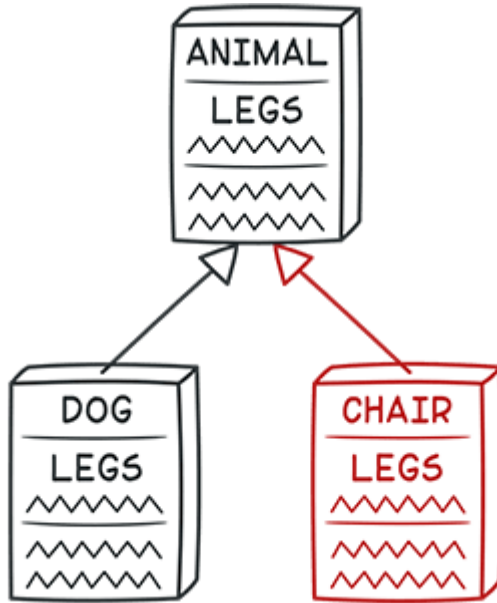
- La utilización inapropiada de principios de la programación orientada a objetos



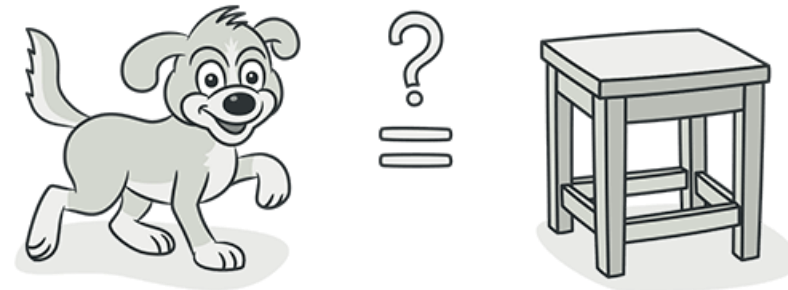


# OBJECT-ORIENTATION ABUSERS

## REFUSED BEQUEST



- **Síntoma:** una clase utiliza algunos de los métodos o atributos heredados, a veces redefiniendo comportamiento
- **Razones:** con el propósito de reutilizar se creó una jerarquía en donde la clase base y las derivadas son distintas
- **Remedio:** eliminar la herencia y utilizar composición (si es innecesaria), o eliminar los métodos no utilizados de la clase derivada (si es necesaria)



# BIBLIOGRAFÍA

- *Code Smells*, de <https://refactoring.guru/refactoring/smells>



# ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 4 – CODE SMELLS

